

Practical 1

1.Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

CODE:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace chrono;

//----- Bubble Sort -----
void bubbleSort(vector<int> &a)
{
    int n = a.size();

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                swap(a[j], a[j + 1]);
            }
        }
    }
}

//----- Selection Sort -----
void selectionSort(vector<int> &a)
{
    int n = a.size();

    for (int i = 0; i < n - 1; i++)
    {
        int min = i;

        for (int j = i + 1; j < n; j++)
        {
            if (a[j] < a[min])
            {
```

```
        min = j;
    }
}

swap(a[i], a[min]);
}
}

//----- Insertion Sort -----
void insertionSort(vector<int> &a)
{
    int n = a.size();

    for (int i = 1; i < n; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

//----- Merge Sort -----
void merge(vector<int> &a, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++)
        L[i] = a[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = a[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            a[k++] = L[i++];
        else
```

```
        a[k++] = R[j++];
    }

    while (i < n1)
        a[k++] = L[i++];

    while (j < n2)
        a[k++] = R[j++];
}

void mergeSort(vector<int> &a, int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);

        merge(a, left, mid, right);
    }
}

//----- Quick Sort -----
int partition(vector<int> &a, int low, int high)
{
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (a[j] < pivot)
        {
            i++;
            swap(a[i], a[j]);
        }
    }

    swap(a[i + 1], a[high]);

    return i + 1;
}

void quickSort(vector<int> &a, int low, int high)
{
    if (low < high)
    {
        int p = partition(a, low, high);
```

```
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

//----- Print Array -----
void printArray(vector<int> a)
{
    for (int x : a)
        cout << x << " ";
    cout << endl;
}

//----- Main Function -----
int main()
{
    int n = 20;

    vector<int> arr(n);

    srand(time(0));

    for (int i = 0; i < n; i++)
    {
        arr[i] = rand() % 100;
    }

    vector<int> temp;

    // Bubble Sort
    temp = arr;
    auto start = high_resolution_clock::now();
    bubbleSort(temp);
    auto stop = high_resolution_clock::now();
    cout << "Bubble Sort Time : "
        << duration_cast<microseconds>(stop - start).count()
        << " microseconds" << endl;

    // Selection Sort
    temp = arr;
    start = high_resolution_clock::now();
    selectionSort(temp);
    stop = high_resolution_clock::now();
    cout << "Selection Sort Time : "
        << duration_cast<microseconds>(stop - start).count()
        << " microseconds" << endl;

    // Insertion Sort
    temp = arr;
```

```
start = high_resolution_clock::now();
insertionSort(temp);
stop = high_resolution_clock::now();
cout << "Insertion Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds" << endl;

// Merge Sort
temp = arr;
start = high_resolution_clock::now();
mergeSort(temp, 0, n - 1);
stop = high_resolution_clock::now();
cout << "Merge Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds" << endl;

// Quick Sort
temp = arr;
start = high_resolution_clock::now();
quickSort(temp, 0, n - 1);
stop = high_resolution_clock::now();
cout << "Quick Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds" << endl;

return 0;
}
```